Diffinite — Source Code Diff Report

■ **Analysis Configuration**
Mode: simple | **Profile:** industrial | **K=**5, **W=**4, **T=**0.10 | **Tokenizer:** token

Dir A: example\aosp\left
Dir B: example\aosp\right
Comparison unit: line | **Comments:** excluded | **Matched pairs:** 3 | **Unmatched:** 0 (A) / 0 (B)

Summary

#	File A	File B	Name Sim.	Content Match	Added	Deleted
1	Handler.java	Handler.java	100.0	88.6%	+144	-76
2	Looper.java	Looper.java	100.0	90.0%	+82	-5
3	Message.java	Message.java	100.0	96.3%	+42	-6

1. Handler.java ↔ Handler.java

Match ratio: 88.6% +144 line(s) -76 line(s)

#	Handler.java	#	Handler.java
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	
8		8	
9		9	
10		10	
11		11	
12		12	
13		13	
14		14	
15		15	
16		16	
17	`package android.os;`	17	`package android.os;`
18		18	
19	`import android.annotation.NonNull;`	19	`import android.annotation.NonNull;`
20	`import android.annotation.Nullable;`	20	`import android.annotation.Nullable;`
		21	`import android.compat.annotation.UnsupportedAppUsage;`
21	`import android.util.Log;`	22	`import android.util.Log;`
22	`import android.util.Printer;`	23	`import android.util.Printer;`
23		24	
24	`import java.lang.reflect.Modifier;`	25	`import java.lang.reflect.Modifier;`
25			
26		26	
27		27	
28		28	
29		29	
30		30	
31		31	
32		32	
33		33	
34		34	
35		35	
36		36	
37		37	
38		38	
39		39	
40		40	

```
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66 public class Handler {
67
68
69
70
71
72 private static final boolean FIND_POTENTIAL_LEAKS = false;
73 private static final String TAG = "Handler";
74 private static Handler MAIN_THREAD_HANDLER = null;
75
76
77
78
79
80 public interface Callback {
81
82
83
84
85 public boolean handleMessage(Message msg);
86 }
87
88
```

```
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66 public class Handler {
67
68
69
70
71
72 private static final boolean FIND_POTENTIAL_LEAKS = false;
73 private static final String TAG = "Handler";
74 private static Handler MAIN_THREAD_HANDLER = null;
75
76
77
78
79
80 public interface Callback {
81
82
83
84
85 boolean handleMessage(@NonNull Message msg);
86 }
87
88
```

```
89
90
91 public void handleMessage(Message msg) {
92 }
93
94
95
96
97 public void dispatchMessage(Message msg) {
98 if (msg.callback != null) {
99 handleCallback(msg);
100 } else {
101 if (mCallback != null) {
102 if (mCallback.handleMessage(msg)) {
103 return;
104 }
105 }
106 handleMessage(msg);
107 }
108 }
109
110
111
112
113
114
115
116
117 public Handler() {
118 this(null, false);
119 }
120
121
122
123
124
125
```

```
89
90
91 public void handleMessage(@NonNull Message msg) {
92 }
93
94
95
96
97 public void dispatchMessage(@NonNull Message msg) {
98 if (msg.callback != null) {
99 handleCallback(msg);
100 } else {
101 if (mCallback != null) {
102 if (mCallback.handleMessage(msg)) {
103 return;
104 }
105 }
106 handleMessage(msg);
107 }
108 }
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127 @Deprecated
128 public Handler() {
129 this(null, false);
130 }
131
132
133
134
135
136
```

```
126
127
128
129
130
131 public Handler(Callback callback) {
132 this(callback, false);
133 }
134
135
136
137
138
139
140 public Handler(Looper looper) {
141 this(looper, null, false);
142 }
143
144
145
146
147
148
149
150
151 public Handler(Looper looper, Callback callback) {
152 this(looper, callback, false);
153 }
154
155
156
157
158
159
160
161
162
163
```

```
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151 @Deprecated
152 public Handler(@Nullable Callback callback) {
153 this(callback, false);
154 }
155
156
157
158
159
160
161 public Handler(@NonNull Looper looper) {
162 this(looper, null, false);
163 }
164
165
166
167
168
169
170
171
172 public Handler(@NonNull Looper looper, @Nullable Callback callback) {
173 this(looper, callback, false);
174 }
175
176
177
178
179
180
181
182
183
184
```

```
164
165
166
167
168
169
170
171 public Handler(boolean async) {
172 this(null, async);
173 }
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192 public Handler(Callback callback, boolean async) {
193 if (FIND_POTENTIAL_LEAKS) {
194 final Class<? extends Handler> klass = getClass();
195 if ((klass.isAnonymousClass() || klass.isMemberClass() || klass.isLocalClass()) &&
196 (klass.getModifiers() & Modifier.STATIC) == 0) {
197 Log.w(TAG, "The following Handler class should be static or leaks might occur: " +
198 klass.getCanonicalName());
199 }
200 }
201
202 mLooper = Looper.myLooper();
203 if (mLooper == null) {
204 throw new RuntimeException(
205 "Can't create handler inside thread " + Thread.currentThread()
206 + " that has not called Looper.prepare()");
207 }
208 mQueue = mLooper.mQueue;
209 mCallback = callback;
210 mAsynchronous = async;
```

```
185
186
187
188
189
190
191
192 @UnsupportedAppUsage
193 public Handler(boolean async) {
194 this(null, async);
195 }
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214 public Handler(@Nullable Callback callback, boolean async) {
215 if (FIND_POTENTIAL_LEAKS) {
216 final Class<? extends Handler> klass = getClass();
217 if ((klass.isAnonymousClass() || klass.isMemberClass() || klass.isLocalClass()) &&
218 (klass.getModifiers() & Modifier.STATIC) == 0) {
219 Log.w(TAG, "The following Handler class should be static or leaks might occur: " +
220 klass.getCanonicalName());
221 }
222 }
223
224 mLooper = Looper.myLooper();
225 if (mLooper == null) {
226 throw new RuntimeException(
227 "Can't create handler inside thread " + Thread.currentThread()
228 + " that has not called Looper.prepare()");
229 }
230 mQueue = mLooper.mQueue;
231 mCallback = callback;
232 mAsynchronous = async;
```

```
211 }
232 public Handler(Looper looper, Callback callback, boolean async) {
233 mLooper = looper;
234 mQueue = looper.mQueue;
235 mCallback = callback;
236 mAsynchronous = async;
237 }
251 @NonNull
252 public static Handler createAsync(@NonNull Looper looper) {
253 if (looper == null) throw new NullPointerException("looper must not be null");
254 return new Handler(looper, null, true);
255 }
```

```
233 }
254 @UnsupportedAppUsage
255 public Handler(@NonNull Looper looper, @Nullable Callback callback, boolean async) {
256 mLooper = looper;
257 mQueue = looper.mQueue;
258 mCallback = callback;
259 mAsynchronous = async;
260 }
274 @NonNull
275 public static Handler createAsync(@NonNull Looper looper) {
276 if (looper == null) throw new NullPointerException("looper must not be null");
277 return new Handler(looper, null, true);
278 }
```

```
@NonNull
public static Handler createAsync(@NonNull Looper looper, @NonNull Callback callback) {
if (looper == null) throw new NullPointerException("looper must not be null");
if (callback == null) throw new NullPointerException("callback must not be null");
return new Handler(looper, callback, true);
}

@NonNull
public static Handler getMain() {
if (MAIN_THREAD_HANDLER == null) {
MAIN_THREAD_HANDLER = new Handler(Looper.getMainLooper());
}
return MAIN_THREAD_HANDLER;
}

@NonNull
public static Handler mainIfNull(@Nullable Handler handler) {
return handler == null ? getMain() : handler;
}

public String getTraceName(Message message) {

final StringBuilder sb = new StringBuilder();
sb.append(getClass().getName()).append(": ");
if (message.callback != null) {
sb.append(message.callback.getClass().getName());
} else {
sb.append("#").append(message.what);
}
```

```
@NonNull
public static Handler createAsync(@NonNull Looper looper, @NonNull Callback callback) {
if (looper == null) throw new NullPointerException("looper must not be null");
if (callback == null) throw new NullPointerException("callback must not be null");
return new Handler(looper, callback, true);
}

@UnsupportedAppUsage
@NonNull
public static Handler getMain() {
if (MAIN_THREAD_HANDLER == null) {
MAIN_THREAD_HANDLER = new Handler(Looper.getMainLooper());
}
return MAIN_THREAD_HANDLER;
}

@NonNull
public static Handler mainIfNull(@Nullable Handler handler) {
return handler == null ? getMain() : handler;
}

@NonNull
public String getTraceName(@NonNull Message message) {
if (message.callback instanceof TraceNameSupplier) {
return ((TraceNameSupplier) message.callback).getTraceName();
}

final StringBuilder sb = new StringBuilder();
sb.append(getClass().getName()).append(": ");
if (message.callback != null) {
sb.append(message.callback.getClass().getName());
} else {
sb.append("#").append(message.what);
}
```

```
300 return sb.toString();
301 }
302
303
304
305
306
307
308
309
310
311 public String getMessageName(Message message) {
312 if (message.callback != null) {
313 return message.callback.getClass().getName();
314 }
315 return "0x" + Integer.toHexString(message.what);
316 }
317
318
319
320
321
322
323 public final Message obtainMessage()
324 {
325 return Message.obtain(this);
326 }
327
328
329
330
331
332
333
334 public final Message obtainMessage(int what)
335 {
336 return Message.obtain(this, what);
337 }
338
339
340
341
342
343
344
```

```
329 return sb.toString();
330 }
331
332
333
334
335
336
337
338
339
340 @NonNull
341 public String getMessageName(@NonNull Message message) {
342 if (message.callback != null) {
343 return message.callback.getClass().getName();
344 }
345 return "0x" + Integer.toHexString(message.what);
346 }
347
348
349
350
351
352
353 @NonNull
354 public final Message obtainMessage()
355 {
356 return Message.obtain(this);
357 }
358
359
360
361
362
363
364
365 @NonNull
366 public final Message obtainMessage(int what)
367 {
368 return Message.obtain(this, what);
369 }
370
371
372
373
374
375
376
```

```
348 public final Message obtainMessage(int what, Object obj)
349 {
350 return Message.obtain(this, what, obj);
351 }
```

```
380 @NonNull
381 public final Message obtainMessage(int what, @Nullable Object obj) {
382 return Message.obtain(this, what, obj);
383 }
```

```
362 public final Message obtainMessage(int what, int arg1, int arg2)
363 {
364 return Message.obtain(this, what, arg1, arg2);
365 }
```

```
394 @NonNull
395 public final Message obtainMessage(int what, int arg1, int arg2)
396 {
397 return Message.obtain(this, what, arg1, arg2);
398 }
```

```
377 public final Message obtainMessage(int what, int arg1, int arg2, Object obj)
378 {
379 return Message.obtain(this, what, arg1, arg2, obj);
380 }
```

```
410 @NonNull
411 public final Message obtainMessage(int what, int arg1, int arg2, @Nullable Object obj) {
412 return Message.obtain(this, what, arg1, arg2, obj);
413 }
```

```
392
393 public final boolean post(Runnable r)
394 {
395 return sendMessageDelayed(getPostMessage(r), 0);
396 }
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416 public final boolean postAtTime(Runnable r, long uptimeMillis)
417 {
418 return sendMessageAtTime(getPostMessage(r), uptimeMillis);
419 }
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
```

```
425
426 public final boolean post(@NonNull Runnable r) {
427 return sendMessageDelayed(getPostMessage(r), 0);
428 }
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448 public final boolean postAtTime(@NonNull Runnable r, long uptimeMillis) {
449 return sendMessageAtTime(getPostMessage(r), uptimeMillis);
450 }
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
```

```
440
441
442
443 public final boolean postAtTime(Runnable r, Object token, long uptimeMillis)
444 {
445 return sendMessageAtTime(getPostMessage(r, token), uptimeMillis);
446 }
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467 public final boolean postDelayed(Runnable r, long delayMillis)
468 {
469 return sendMessageDelayed(getPostMessage(r), delayMillis);
470 }
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
```

```
471
472
473
474 public final boolean postAtTime(
475 @NonNull Runnable r, @Nullable Object token, long uptimeMillis) {
476 return sendMessageAtTime(getPostMessage(r, token), uptimeMillis);
477 }
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498 public final boolean postDelayed(@NonNull Runnable r, long delayMillis) {
499 return sendMessageDelayed(getPostMessage(r), delayMillis);
500 }
501
502
503 public final boolean postDelayed(Runnable r, int what, long delayMillis) {
504 return sendMessageDelayed(getPostMessage(r).setWhat(what), delayMillis);
505 }
506
507
508
509
510
511
512
513
514
515
516
517
```

```
488
489
490
491
492
493 public final boolean postDelayed(Runnable r, Object token, long delayMillis)
494 {
495 return sendMessageDelayed(getPostMessage(r, token), delayMillis);
496 }
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513 public final boolean postAtFrontOfQueue(Runnable r)
514 {
515 return sendMessageAtFrontOfQueue(getPostMessage(r));
516 }
517
518
519
520
521
522
523
524
525
526
527
528
529
530
```

```
518
519
520
521
522
523
524
525
526
527
528 public final boolean postDelayed(
529 @NonNull Runnable r, @Nullable Object token, long delayMillis) {
530 return sendMessageDelayed(getPostMessage(r, token), delayMillis);
531 }
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548 public final boolean postAtFrontOfQueue(@NonNull Runnable r) {
549 return sendMessageAtFrontOfQueue(getPostMessage(r));
550 }
551
552
553
554
555
556
557
558
559
560
561
562
563
564
```

```
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558 public final boolean runWithScissors(final Runnable r, long timeout) {
559 if (r == null) {
560 throw new IllegalArgumentException("runnable must not be null");
561 }
562 if (timeout < 0) {
563 throw new IllegalArgumentException("timeout must be non-negative");
564 }
565
566 if (Looper.myLooper() == mLooper) {
567 r.run();
568 return true;
569 }
570
571 BlockingRunnable br = new BlockingRunnable(r);
572 return br.postAndWait(this, timeout);
573 }
574
575
576
577
578 public final void removeCallbacks(Runnable r)
```

```
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592 public final boolean runWithScissors(@NonNull Runnable r, long timeout) {
593 if (r == null) {
594 throw new IllegalArgumentException("runnable must not be null");
595 }
596 if (timeout < 0) {
597 throw new IllegalArgumentException("timeout must be non-negative");
598 }
599
600 if (Looper.myLooper() == mLooper) {
601 r.run();
602 return true;
603 }
604
605 BlockingRunnable br = new BlockingRunnable(r);
606 return br.postAndWait(this, timeout);
607 }
608
609
610
611
612 public final void removeCallbacks(@NonNull Runnable r) {
```

579	{		
580	mQueue.removeMessages(this, r, null);	613	mQueue.removeMessages(this, r, null);
581	}	614	}
582		615	
583		616	
584		617	
585		618	
586		619	
587		620	
588	public final void removeCallbacks(Runnable r, Object token)	621	public final void removeCallbacks(@NonNull Runnable r, @Nullable Object token) {
589	{		
590	mQueue.removeMessages(this, r, token);	622	mQueue.removeMessages(this, r, token);
591	}	623	}
592		624	
593		625	
594		626	
595		627	
596		628	
597		629	
598		630	
599		631	
600		632	
601		633	
602	public final boolean sendMessage(Message msg)	634	public final boolean sendMessage(@NonNull Message msg) {
603	{		
604	return sendMessageDelayed(msg, 0);	635	return sendMessageDelayed(msg, 0);
605	}	636	}
606		637	
607		638	
608		639	
609		640	
610		641	
611		642	
612		643	
613		644	
614	public final boolean sendEmptyMessage(int what)	645	public final boolean sendEmptyMessage(int what)
615	{	646	{
616	return sendEmptyMessageDelayed(what, 0);	647	return sendEmptyMessageDelayed(what, 0);
617	}	648	}
618		649	
619		650	
620		651	
621		652	
622		653	
623		654	
624		655	
625		656	
626		657	

```
627
628 public final boolean sendEmptyMessageDelayed(int what, long delayMillis) {
629 Message msg = Message.obtain();
630 msg.what = what;
631 return sendMessageDelayed(msg, delayMillis);
632 }
633
634
635
636
637
638
639
640
641
642
643
644 public final boolean sendEmptyMessageAtTime(int what, long uptimeMillis) {
645 Message msg = Message.obtain();
646 msg.what = what;
647 return sendMessageAtTime(msg, uptimeMillis);
648 }
649
650
651
652
653
654
655
656
657
658
659
660
661
662 public final boolean sendMessageDelayed(Message msg, long delayMillis)
663 {
664 if (delayMillis < 0) {
665 delayMillis = 0;
666 }
667 return sendMessageAtTime(msg, SystemClock.uptimeMillis() + delayMillis);
668 }
669
670
671
672
673
674
```

```
658
659 public final boolean sendEmptyMessageDelayed(int what, long delayMillis) {
660 Message msg = Message.obtain();
661 msg.what = what;
662 return sendMessageDelayed(msg, delayMillis);
663 }
664
665
666
667
668
669
670
671
672
673
674
675 public final boolean sendEmptyMessageAtTime(int what, long uptimeMillis) {
676 Message msg = Message.obtain();
677 msg.what = what;
678 return sendMessageAtTime(msg, uptimeMillis);
679 }
680
681
682
683
684
685
686
687
688
689
690
691
692
693 public final boolean sendMessageDelayed(@NonNull Message msg, long delayMillis) {

694 if (delayMillis < 0) {
695 delayMillis = 0;
696 }
697 return sendMessageAtTime(msg, SystemClock.uptimeMillis() + delayMillis);
698 }
699
700
701
702
703
704
```

```
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689 public boolean sendMessageAtTime(Message msg, long uptimeMillis) {
690 MessageQueue queue = mQueue;
691 if (queue == null) {
692 RuntimeException e = new RuntimeException(
693 this + " sendMessageAtTime() called with no mQueue");
694 Log.w("Looper", e.getMessage(), e);
695 return false;
696 }
697 return enqueueMessage(queue, msg, uptimeMillis);
698 }
699
700
701
702
703
704
705
706
707
708
709
710
711
712 public final boolean sendMessageAtFrontOfQueue(Message msg) {
713 MessageQueue queue = mQueue;
714 if (queue == null) {
715 RuntimeException e = new RuntimeException(
716 this + " sendMessageAtTime() called with no mQueue");
717 Log.w("Looper", e.getMessage(), e);
718 return false;
719 }
720 return enqueueMessage(queue, msg, 0);
721 }
722
```

```
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719 public boolean sendMessageAtTime(@NonNull Message msg, long uptimeMillis) {
720 MessageQueue queue = mQueue;
721 if (queue == null) {
722 RuntimeException e = new RuntimeException(
723 this + " sendMessageAtTime() called with no mQueue");
724 Log.w("Looper", e.getMessage(), e);
725 return false;
726 }
727 return enqueueMessage(queue, msg, uptimeMillis);
728 }
729
730
731
732
733
734
735
736
737
738
739
740
741
742 public final boolean sendMessageAtFrontOfQueue(@NonNull Message msg) {
743 MessageQueue queue = mQueue;
744 if (queue == null) {
745 RuntimeException e = new RuntimeException(
746 this + " sendMessageAtTime() called with no mQueue");
747 Log.w("Looper", e.getMessage(), e);
748 return false;
749 }
750 return enqueueMessage(queue, msg, 0);
751 }
752
```

```
723
724
725
726
727
728
729
730
731
732 public final boolean executeOrSendMessage(Message msg) {
733 if (mLooper == Looper.myLooper()) {
734 dispatchMessage(msg);
735 return true;
736 }
737 return sendMessage(msg);
738 }
739
740 private boolean enqueueMessage(MessageQueue queue, Message msg, long uptimeMillis) {
741 msg.target = this;
742 if (mAsynchronous) {
743 msg.setAsynchronous(true);
744 }
745 return queue.enqueueMessage(msg, uptimeMillis);
746 }
747
748
749
750
751
752 public final void removeMessages(int what) {
753 mQueue.removeMessages(this, what, null);
754 }
755
756
757
758
759
760
761 public final void removeMessages(int what, Object object) {
762 mQueue.removeMessages(this, what, object);
763 }
764
765
766
767
```

```
753
754
755
756
757
758
759
760
761
762 public final boolean executeOrSendMessage(@NonNull Message msg) {
763 if (mLooper == Looper.myLooper()) {
764 dispatchMessage(msg);
765 return true;
766 }
767 return sendMessage(msg);
768 }
769
770 private boolean enqueueMessage(@NonNull MessageQueue queue, @NonNull Message msg,
771 long uptimeMillis) {
772 msg.target = this;
773 msg.workSourceUid = ThreadLocalWorkSource.getUid();
774
775 if (mAsynchronous) {
776 msg.setAsynchronous(true);
777 }
778 return queue.enqueueMessage(msg, uptimeMillis);
779 }
780
781
782
783
784
785 public final void removeMessages(int what) {
786 mQueue.removeMessages(this, what, null);
787 }
788
789
790
791
792
793
794 public final void removeMessages(int what, @Nullable Object object) {
795 mQueue.removeMessages(this, what, object);
796 }
797
798
799
800
```

```
768
769
770 public final void removeCallbacksAndMessages(Object token) {
771 mQueue.removeCallbacksAndMessages(this, token);
772 }
773
774
775
776
777
778 public final boolean hasMessages(int what) {
779 return mQueue.hasMessages(this, what, null);
780 }
781
782
783
784
785
786 public final boolean hasMessagesOrCallbacks() {
787 return mQueue.hasMessages(this);
788 }
789
790
791
792
793
794 public final boolean hasMessages(int what, Object object) {
```

```
801
802
803
804
805 public final void removeEqualMessages(int what, @Nullable Object object) {
806 mQueue.removeEqualMessages(this, what, object);
807 }
808
809
810
811
812
813
814 public final void removeCallbacksAndMessages(@Nullable Object token) {
815 mQueue.removeCallbacksAndMessages(this, token);
816 }
817
818
819
820
821
822
823
824
825 public final void removeCallbacksAndEqualMessages(@Nullable Object token) {
826 mQueue.removeCallbacksAndEqualMessages(this, token);
827 }
828
829
830
831
832 public final boolean hasMessages(int what) {
833 return mQueue.hasMessages(this, what, null);
834 }
835
836
837
838
839
840 public final boolean hasMessagesOrCallbacks() {
841 return mQueue.hasMessages(this);
842 }
843
844
845
846
847
848 public final boolean hasMessages(int what, @Nullable Object object) {
```

```
795 return mQueue.hasMessages(this, what, object);
796 }
797
798
799
800
801
802
803
804 public final boolean hasCallbacks(Runnable r) {
805 return mQueue.hasMessages(this, r, null);
806 }
807
808
809
810 public final Looper getLooper() {
811 return mLooper;
812 }
813
814 public final void dump(Printer pw, String prefix) {
815 pw.println(prefix + this + " @ " + SystemClock.uptimeMillis());
816 if (mLooper == null) {
817 pw.println(prefix + "looper uninitialized");
818 } else {
819 mLooper.dump(pw, prefix + " ");
820 }
821 }
822
823
824
825
826 public final void dumpMine(Printer pw, String prefix) {
827 pw.println(prefix + this + " @ " + SystemClock.uptimeMillis());
828 if (mLooper == null) {
829 pw.println(prefix + "looper uninitialized");
830 } else {
831 mLooper.dump(pw, prefix + " ", this);
832 }
833 }
```

```
849 return mQueue.hasMessages(this, what, object);
850 }
851
852
853
854
855
856
857
858 public final boolean hasEqualMessages(int what, @Nullable Object object) {
859 return mQueue.hasEqualMessages(this, what, object);
860 }
861
862
863
864
865
866 public final boolean hasCallbacks(@NonNull Runnable r) {
867 return mQueue.hasMessages(this, r, null);
868 }
869
870
871
872 @NonNull
873 public final Looper getLooper() {
874 return mLooper;
875 }
876
877 public final void dump(@NonNull Printer pw, @NonNull String prefix) {
878 pw.println(prefix + this + " @ " + SystemClock.uptimeMillis());
879 if (mLooper == null) {
880 pw.println(prefix + "looper uninitialized");
881 } else {
882 mLooper.dump(pw, prefix + " ");
883 }
884 }
885
886
887
888
889 public final void dumpMine(@NonNull Printer pw, @NonNull String prefix) {
890 pw.println(prefix + this + " @ " + SystemClock.uptimeMillis());
891 if (mLooper == null) {
892 pw.println(prefix + "looper uninitialized");
893 } else {
894 mLooper.dump(pw, prefix + " ", this);
895 }
896 }
```

```
834
835 @Override
836 public String toString() {
837 return "Handler (" + getClass().getName() + ") {"
838 + Integer.toHexString(System.identityHashCode(this))
839 + "}";
840 }
841
842 final IMessenger getIMessenger() {
843 synchronized (mQueue) {
844 if (mMessenger != null) {
845 return mMessenger;
846 }
847 mMessenger = new MessengerImpl();
848 return mMessenger;
849 }
850 }
851
852 private final class MessengerImpl extends IMessenger.Stub {
853 public void send(Message msg) {
854 msg.sendingUid = Binder.getCallingUid();
855 Handler.this.sendMessage(msg);
856 }
857 }
858
859 private static Message getPostMessage(Runnable r) {
860 Message m = Message.obtain();
861 m.callback = r;
862 return m;
863 }
864
865 private static Message getPostMessage(Runnable r, Object token) {
866 Message m = Message.obtain();
867 m.obj = token;
868 m.callback = r;
869 return m;
870 }
871
872 private static void handleCallback(Message message) {
873 message.callback.run();
874 }
875
876 final Looper mLooper;
877 final MessageQueue mQueue;
```

```
897
898 @Override
899 public String toString() {
900 return "Handler (" + getClass().getName() + ") {"
901 + Integer.toHexString(System.identityHashCode(this))
902 + "}";
903 }
904
905 @UnsupportedAppUsage
906 final IMessenger getIMessenger() {
907 synchronized (mQueue) {
908 if (mMessenger != null) {
909 return mMessenger;
910 }
911 mMessenger = new MessengerImpl();
912 return mMessenger;
913 }
914 }
915
916 private final class MessengerImpl extends IMessenger.Stub {
917 public void send(Message msg) {
918 msg.sendingUid = Binder.getCallingUid();
919 Handler.this.sendMessage(msg);
920 }
921 }
922
923 private static Message getPostMessage(Runnable r) {
924 Message m = Message.obtain();
925 m.callback = r;
926 return m;
927 }
928
929 @UnsupportedAppUsage
930 private static Message getPostMessage(Runnable r, Object token) {
931 Message m = Message.obtain();
932 m.obj = token;
933 m.callback = r;
934 return m;
935 }
936
937 private static void handleCallback(Message message) {
938 message.callback.run();
939 }
940
941 @UnsupportedAppUsage
942 final Looper mLooper;
943 final MessageQueue mQueue;
944 @UnsupportedAppUsage
```

```
878 final Callback mCallback;
879 final boolean mAsynchronous;
880 IMessenger mMessenger;
881
882 private static final class BlockingRunnable implements Runnable {
883 private final Runnable mTask;
884 private boolean mDone;
885
886 public BlockingRunnable(Runnable task) {
887 mTask = task;
888 }
889
890 @Override
891 public void run() {
892 try {
893 mTask.run();
894 } finally {
895 synchronized (this) {
896 mDone = true;
897 notifyAll();
898 }
899 }
900 }
901
902 public boolean postAndWait(Handler handler, long timeout) {
903 if (!handler.post(this)) {
904 return false;
905 }
906
907 synchronized (this) {
908 if (timeout > 0) {
909 final long expirationTime = SystemClock.uptimeMillis() + timeout;
910 while (!mDone) {
911 long delay = expirationTime - SystemClock.uptimeMillis();
912 if (delay <= 0) {
913 return false;
914 }
915 try {
916 wait(delay);
917 } catch (InterruptedException ex) {
918 }
919 }
920 } else {
921 while (!mDone) {
922 try {
923 wait();
924 } catch (InterruptedException ex) {
```

```
945 final Callback mCallback;
946 final boolean mAsynchronous;
947 @UnsupportedAppUsage
948 IMessenger mMessenger;
949
950 private static final class BlockingRunnable implements Runnable {
951 private final Runnable mTask;
952 private boolean mDone;
953
954 public BlockingRunnable(Runnable task) {
955 mTask = task;
956 }
957
958 @Override
959 public void run() {
960 try {
961 mTask.run();
962 } finally {
963 synchronized (this) {
964 mDone = true;
965 notifyAll();
966 }
967 }
968 }
969
970 public boolean postAndWait(Handler handler, long timeout) {
971 if (!handler.post(this)) {
972 return false;
973 }
974
975 synchronized (this) {
976 if (timeout > 0) {
977 final long expirationTime = SystemClock.uptimeMillis() + timeout;
978 while (!mDone) {
979 long delay = expirationTime - SystemClock.uptimeMillis();
980 if (delay <= 0) {
981 return false;
982 }
983 try {
984 wait(delay);
985 } catch (InterruptedException ex) {
986 }
987 }
988 } else {
989 while (!mDone) {
990 try {
991 wait();
992 } catch (InterruptedException ex) {
```

```
}
}
}
}
return true;
}
}
}
```

```
}
}
}
}
return true;
}
}
}
```

2. Looper.java ↔ Looper.java

Match ratio: 90.0% +82 line(s) -5 line(s)

#	Looper.java	#	Looper.java
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	
8		8	
9		9	
10		10	
11		11	
12		12	
13		13	
14		14	
15		15	
16		16	
17	`package android.os;`	17	`package android.os;`
18		18	
19	`import android.annotation.NonNull;`	19	`import android.annotation.NonNull;`
20	`import android.annotation.Nullable;`	20	`import android.annotation.Nullable;`
21	`import android.os.LooperProto;`	21	`import android.compat.annotation.UnsupportedAppUsage;`
22	`import android.util.Log;`	22	`import android.util.Log;`
23	`import android.util.Printer;`	23	`import android.util.Printer;`
24	`import android.util.Slog;`	24	`import android.util.Slog;`
25	`import android.util.proto.ProtoOutputStream;`	25	`import android.util.proto.ProtoOutputStream;`
26		26	
27		27	
28		28	
29		29	
30		30	
31		31	
32		32	
33		33	
34		34	
35		35	
36		36	
37		37	
38		38	
39		39	
40		40	
41		41	

```
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57 public final class Looper {
58
59
60
61
62
63
64
65
66
67
68 private static final String TAG = "Looper";
69
70
71 static final ThreadLocal<Looper> sThreadLocal = new ThreadLocal<Looper>();
72 private static Looper sMainLooper;
73
74 final MessageQueue mQueue;
75 final Thread mThread;
76
77 private Printer mLogging;
78 private long mTraceTag;
79
80
81
82
83 private long mSlowDispatchThresholdMs;
```

```
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57 public final class Looper {
58
59
60
61
62
63
64
65
66
67
68 private static final String TAG = "Looper";
69
70
71 @UnsupportedAppUsage
72 static final ThreadLocal<Looper> sThreadLocal = new ThreadLocal<Looper>();
73 @UnsupportedAppUsage
74 private static Looper sMainLooper;
75 private static Observer sObserver;
76
77 @UnsupportedAppUsage
78 final MessageQueue mQueue;
79 final Thread mThread;
80 private boolean mInLoop;
81
82 @UnsupportedAppUsage
83 private Printer mLogging;
84 private long mTraceTag;
85
86
87
88
89 private long mSlowDispatchThresholdMs;
```

```
84
85
86
87
88
89 private long mSlowDeliveryThresholdMs;
90
91
92
93
94
95
96
97 public static void prepare() {
98 prepare(true);
99 }
100
101 private static void prepare(boolean quitAllowed) {
102 if (sThreadLocal.get() != null) {
103 throw new RuntimeException("Only one Looper may be created per thread");
104 }
105 sThreadLocal.set(new Looper(quitAllowed));
106 }
107
108
109
110
111
112
113
114 public static void prepareMainLooper() {
115 prepare(false);
116 synchronized (Looper.class) {
117 if (sMainLooper != null) {
118 throw new IllegalStateException("The main Looper has already been prepared.");
119 }
120 sMainLooper = myLooper();
121 }
122 }
123
124
125
126
127 public static Looper getMainLooper() {
128 synchronized (Looper.class) {
129 return sMainLooper;
```

```
90
91
92
93
94
95 private long mSlowDeliveryThresholdMs;
96
97
98
99
100
101
102
103 public static void prepare() {
104 prepare(true);
105 }
106
107 private static void prepare(boolean quitAllowed) {
108 if (sThreadLocal.get() != null) {
109 throw new RuntimeException("Only one Looper may be created per thread");
110 }
111 sThreadLocal.set(new Looper(quitAllowed));
112 }
113
114
115
116
117
118
119
120
121 @Deprecated
122 public static void prepareMainLooper() {
123 prepare(false);
124 synchronized (Looper.class) {
125 if (sMainLooper != null) {
126 throw new IllegalStateException("The main Looper has already been prepared.");
127 }
128 sMainLooper = myLooper();
129 }
130 }
131
132
133
134
135 public static Looper getMainLooper() {
136 synchronized (Looper.class) {
137 return sMainLooper;
```

```
130 }
131 }
132
133
134
135
136
137 public static void loop() {
138 final Looper me = myLooper();
139 if (me == null) {
140 throw new RuntimeException("No Looper; Looper.prepare() wasn't called on this thread.");
141 }
142 final MessageQueue queue = me.mQueue;
143
144
145
146 Binder.clearCallingIdentity();
147 final long ident = Binder.clearCallingIdentity();
148
149
150
151 final int thresholdOverride =
152 SystemProperties.getInt("log.looper."
153 + Process.myUid() + "."
154 + Thread.currentThread().getName()
155 + ".slow", 0);
156
157 boolean slowDeliveryDetected = false;
158
159 for (;;) {
160 Message msg = queue.next();
161 if (msg == null) {
162
```

```
138 }
139 }
140
141
142
143
144
145
146 public static void setObserver(@Nullable Observer observer) {
147 sObserver = observer;
148 }
149
150
151
152
153
154 public static void loop() {
155 final Looper me = myLooper();
156 if (me == null) {
157 throw new RuntimeException("No Looper; Looper.prepare() wasn't called on this thread.");
158 }
159 if (me.mInLoop) {
160 Slog.w(TAG, "Loop again would have the queued messages be executed"
161 + " before this one completed.");
162 }
163
164 me.mInLoop = true;
165 final MessageQueue queue = me.mQueue;
166
167
168
169 Binder.clearCallingIdentity();
170 final long ident = Binder.clearCallingIdentity();
171
172
173
174 final int thresholdOverride =
175 SystemProperties.getInt("log.looper."
176 + Process.myUid() + "."
177 + Thread.currentThread().getName()
178 + ".slow", 0);
179
180 boolean slowDeliveryDetected = false;
181
182 for (;;) {
183 Message msg = queue.next();
184 if (msg == null) {
185
```

```
163 return;
164 }
165
166
167 final Printer logging = me.mLogging;
168 if (logging != null) {
169 logging.println(">>>>> Dispatching to " + msg.target + " " +
170 msg.callback + ": " + msg.what);
171 }
172
173 final long traceTag = me.mTraceTag;
174 long slowDispatchThresholdMs = me.mSlowDispatchThresholdMs;
175 long slowDeliveryThresholdMs = me.mSlowDeliveryThresholdMs;
176 if (thresholdOverride > 0) {
177 slowDispatchThresholdMs = thresholdOverride;
178 slowDeliveryThresholdMs = thresholdOverride;
179 }
180 final boolean logSlowDelivery = (slowDeliveryThresholdMs > 0) && (msg.when > 0);
181 final boolean logSlowDispatch = (slowDispatchThresholdMs > 0);
182
183 final boolean needStartTime = logSlowDelivery || logSlowDispatch;
184 final boolean needEndTime = logSlowDispatch;
185
186 if (traceTag != 0 && Trace.isTagEnabled(traceTag)) {
187 Trace.traceBegin(traceTag, msg.target.getTraceName(msg));
188 }
189
190 final long dispatchStart = needStartTime ? SystemClock.uptimeMillis() : 0;
191 final long dispatchEnd;
192 try {
193 msg.target.dispatchMessage(msg);
194 dispatchEnd = needEndTime ? SystemClock.uptimeMillis() : 0;
195 } finally {
```

```
186 return;
187 }
188
189
190 final Printer logging = me.mLogging;
191 if (logging != null) {
192 logging.println(">>>>> Dispatching to " + msg.target + " " +
193 msg.callback + ": " + msg.what);
194 }
195
196 final Observer observer = sObserver;
197
198 final long traceTag = me.mTraceTag;
199 long slowDispatchThresholdMs = me.mSlowDispatchThresholdMs;
200 long slowDeliveryThresholdMs = me.mSlowDeliveryThresholdMs;
201 if (thresholdOverride > 0) {
202 slowDispatchThresholdMs = thresholdOverride;
203 slowDeliveryThresholdMs = thresholdOverride;
204 }
205 final boolean logSlowDelivery = (slowDeliveryThresholdMs > 0) && (msg.when > 0);
206 final boolean logSlowDispatch = (slowDispatchThresholdMs > 0);
207
208 final boolean needStartTime = logSlowDelivery || logSlowDispatch;
209 final boolean needEndTime = logSlowDispatch;
210
211 if (traceTag != 0 && Trace.isTagEnabled(traceTag)) {
212 Trace.traceBegin(traceTag, msg.target.getTraceName(msg));
213 }
214
215 final long dispatchStart = needStartTime ? SystemClock.uptimeMillis() : 0;
216 final long dispatchEnd;
217 Object token = null;
218 if (observer != null) {
219 token = observer.messageDispatchStarting();
220 }
221 long origWorkSource = ThreadLocalWorkSource.setUid(msg.workSourceUid);
222 try {
223 msg.target.dispatchMessage(msg);
224 if (observer != null) {
225 observer.messageDispatched(token, msg);
226 }
227 dispatchEnd = needEndTime ? SystemClock.uptimeMillis() : 0;
228 } catch (Exception exception) {
229 if (observer != null) {
230 observer.dispatchingThrewException(token, msg, exception);
231 }
232 throw exception;
233 } finally {
```

```
196 if (traceTag != 0) {
197 Trace.traceEnd(traceTag);
198 }
199 }
200 if (logSlowDelivery) {
201 if (slowDeliveryDetected) {
202 if ((dispatchStart - msg.when) <= 10) {
203 Slog.w(TAG, "Drained");
204 slowDeliveryDetected = false;
205 }
206 } else {
207 if (showSlowLog(slowDeliveryThresholdMs, msg.when, dispatchStart, "delivery",
208 msg)) {
209
210 slowDeliveryDetected = true;
211 }
212 }
213 }
214 if (logSlowDispatch) {
215 showSlowLog(slowDispatchThresholdMs, dispatchStart, dispatchEnd, "dispatch", msg);
216 }
217
218 if (logging != null) {
219 logging.println("<<<<< Finished to " + msg.target + " " + msg.callback);
220 }
221
222
223
224 final long newIdent = Binder.clearCallingIdentity();
225 if (ident != newIdent) {
226 Log.wtf(TAG, "Thread identity changed from 0x"
227 + Long.toHexString(ident) + " to 0x"
228 + Long.toHexString(newIdent) + " while dispatching to "
229 + msg.target.getClass().getName() + " "
230 + msg.callback + " what=" + msg.what);
231 }
232
233 msg.recycleUnchecked();
234 }
235 }
236
237 private static boolean showSlowLog(long threshold, long measureStart, long measureEnd,
238 String what, Message msg) {
239 final long actualTime = measureEnd - measureStart;
240 if (actualTime < threshold) {
241 return false;
242 }
```

```
234 ThreadLocalWorkSource.restore(origWorkSource);
235 if (traceTag != 0) {
236 Trace.traceEnd(traceTag);
237 }
238 }
239 if (logSlowDelivery) {
240 if (slowDeliveryDetected) {
241 if ((dispatchStart - msg.when) <= 10) {
242 Slog.w(TAG, "Drained");
243 slowDeliveryDetected = false;
244 }
245 } else {
246 if (showSlowLog(slowDeliveryThresholdMs, msg.when, dispatchStart, "delivery",
247 msg)) {
248
249 slowDeliveryDetected = true;
250 }
251 }
252 }
253 if (logSlowDispatch) {
254 showSlowLog(slowDispatchThresholdMs, dispatchStart, dispatchEnd, "dispatch", msg);
255 }
256
257 if (logging != null) {
258 logging.println("<<<<< Finished to " + msg.target + " " + msg.callback);
259 }
260
261
262
263 final long newIdent = Binder.clearCallingIdentity();
264 if (ident != newIdent) {
265 Log.wtf(TAG, "Thread identity changed from 0x"
266 + Long.toHexString(ident) + " to 0x"
267 + Long.toHexString(newIdent) + " while dispatching to "
268 + msg.target.getClass().getName() + " "
269 + msg.callback + " what=" + msg.what);
270 }
271
272 msg.recycleUnchecked();
273 }
274 }
275
276 private static boolean showSlowLog(long threshold, long measureStart, long measureEnd,
277 String what, Message msg) {
278 final long actualTime = measureEnd - measureStart;
279 if (actualTime < threshold) {
280 return false;
281 }
```

```
243
244 Slog.w(TAG, "Slow " + what + " took " + actualTime + "ms "
245 + Thread.currentThread().getName() + " h="
246 + msg.target.getClass().getName() + " c=" + msg.callback + " m=" + msg.what);
247 return true;
248 }
249
250
251
252
253
254 public static @Nullable Looper myLooper() {
255 return sThreadLocal.get();
256 }
257
258
259
260
261
262
263 public static @NonNull MessageQueue myQueue() {
264 return myLooper().mQueue;
265 }
266
267 private Looper(boolean quitAllowed) {
268 mQueue = new MessageQueue(quitAllowed);
269 mThread = Thread.currentThread();
270 }
271
272
273
274
275 public boolean isCurrentThread() {
276 return Thread.currentThread() == mThread;
277 }
278
279
280
281
282
283
284
285
286
287
288 public void setMessageLogging(@Nullable Printer printer) {
289 mLogging = printer;
290 }
```

```
282
283 Slog.w(TAG, "Slow " + what + " took " + actualTime + "ms "
284 + Thread.currentThread().getName() + " h="
285 + msg.target.getClass().getName() + " c=" + msg.callback + " m=" + msg.what);
286 return true;
287 }
288
289
290
291
292
293 public static @Nullable Looper myLooper() {
294 return sThreadLocal.get();
295 }
296
297
298
299
300
301
302 public static @NonNull MessageQueue myQueue() {
303 return myLooper().mQueue;
304 }
305
306 private Looper(boolean quitAllowed) {
307 mQueue = new MessageQueue(quitAllowed);
308 mThread = Thread.currentThread();
309 }
310
311
312
313
314 public boolean isCurrentThread() {
315 return Thread.currentThread() == mThread;
316 }
317
318
319
320
321
322
323
324
325
326
327 public void setMessageLogging(@Nullable Printer printer) {
328 mLogging = printer;
329 }
```

```
291
292
293 public void setTraceTag(long traceTag) {
294 mTraceTag = traceTag;
295 }
296
297
298
299
300
301 public void setSlowLogThresholdMs(long slowDispatchThresholdMs, long slowDeliveryThresholdMs) {
302 mSlowDispatchThresholdMs = slowDispatchThresholdMs;
303 mSlowDeliveryThresholdMs = slowDeliveryThresholdMs;
304 }
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322 public void quit() {
323 mQueue.quit(false);
324 }
325
326
327
328
329
330
331
332
333
334
335
336
337
```

```
330
331
332 @UnsupportedAppUsage
333 public void setTraceTag(long traceTag) {
334 mTraceTag = traceTag;
335 }
336
337
338
339
340
341 public void setSlowLogThresholdMs(long slowDispatchThresholdMs, long slowDeliveryThresholdMs) {
342 mSlowDispatchThresholdMs = slowDispatchThresholdMs;
343 mSlowDeliveryThresholdMs = slowDeliveryThresholdMs;
344 }
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362 public void quit() {
363 mQueue.quit(false);
364 }
365
366
367
368
369
370
371
372
373
374
375
376
377
```

```
338 public void quitSafely() {
339 mQueue.quit(true);
340 }
341
342
343
344
345
346
347 public @NonNull Thread getThread() {
348 return mThread;
349 }
350
351
352
353
354
355
356 public @NonNull MessageQueue getQueue() {
357 return mQueue;
358 }
359
360
361
362
363
364
365
366 public void dump(@NonNull Printer pw, @NonNull String prefix) {
367 pw.println(prefix + toString());
368 mQueue.dump(pw, prefix + " ", null);
369 }
370
371
372
373
374
375
376
377
378
379 public void dump(@NonNull Printer pw, @NonNull String prefix, Handler handler) {
380 pw.println(prefix + toString());
381 mQueue.dump(pw, prefix + " ", handler);
382 }
383
384
385 public void writeToProto(ProtoOutputStream proto, long fieldId) {
```

```
378 public void quitSafely() {
379 mQueue.quit(true);
380 }
381
382
383
384
385
386
387 public @NonNull Thread getThread() {
388 return mThread;
389 }
390
391
392
393
394
395
396 public @NonNull MessageQueue getQueue() {
397 return mQueue;
398 }
399
400
401
402
403
404
405
406 public void dump(@NonNull Printer pw, @NonNull String prefix) {
407 pw.println(prefix + toString());
408 mQueue.dump(pw, prefix + " ", null);
409 }
410
411
412
413
414
415
416
417
418
419 public void dump(@NonNull Printer pw, @NonNull String prefix, Handler handler) {
420 pw.println(prefix + toString());
421 mQueue.dump(pw, prefix + " ", handler);
422 }
423
424
425 public void dumpDebug(ProtoOutputStream proto, long fieldId) {
```

```
386 final long looperToken = proto.start(fieldId);
387 proto.write(LooperProto.THREAD_NAME, mThread.getName());
388 proto.write(LooperProto.THREAD_ID, mThread.getId());
389 mQueue.writeToProto(proto, LooperProto.QUEUE);
390 proto.end(looperToken);
391 }
392
393 @Override
394 public String toString() {
395 return "Looper (" + mThread.getName() + ", tid " + mThread.getId()
396 + ") {" + Integer.toHexString(System.identityHashCode(this)) + "}";
397 }
```

```
426 final long looperToken = proto.start(fieldId);
427 proto.write(LooperProto.THREAD_NAME, mThread.getName());
428 proto.write(LooperProto.THREAD_ID, mThread.getId());
429 if (mQueue != null) {
430 mQueue.dumpDebug(proto, LooperProto.QUEUE);
431 }
432 proto.end(looperToken);
433 }
434
435 @Override
436 public String toString() {
437 return "Looper (" + mThread.getName() + ", tid " + mThread.getId()
438 + ") {" + Integer.toHexString(System.identityHashCode(this)) + "}";
439 }
440
441
442 public interface Observer {
443
444
445
446
447
448
449
450
451
452
453
454 Object messageDispatchStarting();
455
456
457
458
459
460
461
462
463 void messageDispatched(Object token, Message msg);
464
465
466
467
468
469
470
471
472
473 void dispatchingThrewException(Object token, Message msg, Exception exception);
```

		474	}
398	}	475	}

3. Message.java ↔ Message.java

Match ratio: 96.3% +42 line(s) -6 line(s)

#	Message.java	#	Message.java
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	
8		8	
9		9	
10		10	
11		11	
12		12	
13		13	
14		14	
15		15	
16		16	
17	`package android.os;`	17	`package android.os;`
18		18	
19	`import android.os.MessageProto;`	19	`import android.compat.annotation.UnsupportedAppUsage;`
20	`import android.util.TimeUtils;`	20	`import android.util.TimeUtils;`
21	`import android.util.proto.ProtoOutputStream;`	21	`import android.util.proto.ProtoOutputStream;`
22		22	
		23	`import com.android.internal.annotations.VisibleForTesting;`
		24	
23		25	
24		26	
25		27	
26		28	
27		29	
28		30	
29		31	
30		32	
31		33	
32		34	
33		35	
34	`public final class Message implements Parcelable {`	36	`public final class Message implements Parcelable {`
35		37	
36		38	
37		39	
38		40	
39		41	

```
40
41 public int what;
42
43
44
45
46
47
48 public int arg1;
49
50
51
52
53
54
55 public int arg2;
56
57
58
59
60
61
62
63
64
65
66
67 public Object obj;
68
69
70
71
72
73
74 public Messenger replyTo;
75
76
77
78
79
80
81 public int sendingUid = -1;
```

```
42
43 public int what;
44
45
46
47
48
49
50 public int arg1;
51
52
53
54
55
56
57 public int arg2;
58
59
60
61
62
63
64
65
66
67
68
69 public Object obj;
70
71
72
73
74
75
76 public Messenger replyTo;
77
78
79
80
81
82
83 public static final int UID_NONE = -1;
84
85
86
87
88
89
```

```
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
```
```
public int sendingUid = UID_NONE;

public int workSourceUid = UID_NONE;

static final int FLAG_IN_USE = 1 << 0;

static final int FLAG_ASYNCHRONOUS = 1 << 1;

static final int FLAGS_TO_CLEAR_ON_COPY_FROM = FLAG_IN_USE;

@UnsupportedAppUsage
int flags;

long when;

@UnsupportedAppUsage
@VisibleForTesting(visibility = VisibleForTesting.Visibility.PACKAGE)
public long when;

Bundle data;

@UnsupportedAppUsage
Handler target;

@UnsupportedAppUsage
Runnable callback;

@UnsupportedAppUsage
Message next;
```

```
111
112
113
114 public static final Object sPoolSync = new Object();
115 private static Message sPool;
116 private static int sPoolSize = 0;
117
118 private static final int MAX_POOL_SIZE = 50;
119
120 private static boolean gCheckRecycle = true;
121
122
123
124
125
126 public static Message obtain() {
127 synchronized (sPoolSync) {
128 if (sPool != null) {
129 Message m = sPool;
130 sPool = m.next;
131 m.next = null;
132 m.flags = 0;
133 sPoolSize--;
134 return m;
135 }
136 }
137 return new Message();
138 }
139
140
141
142
143
144
145
146 public static Message obtain(Message orig) {
147 Message m = obtain();
148 m.what = orig.what;
149 m.arg1 = orig.arg1;
150 m.arg2 = orig.arg2;
151 m.obj = orig.obj;
152 m.replyTo = orig.replyTo;
153 m.sendingUid = orig.sendingUid;
154 if (orig.data != null) {
155 m.data = new Bundle(orig.data);
156 }
157 m.target = orig.target;
```

```
138
139
140
141 public static final Object sPoolSync = new Object();
142 private static Message sPool;
143 private static int sPoolSize = 0;
144
145 private static final int MAX_POOL_SIZE = 50;
146
147 private static boolean gCheckRecycle = true;
148
149
150
151
152
153 public static Message obtain() {
154 synchronized (sPoolSync) {
155 if (sPool != null) {
156 Message m = sPool;
157 sPool = m.next;
158 m.next = null;
159 m.flags = 0;
160 sPoolSize--;
161 return m;
162 }
163 }
164 return new Message();
165 }
166
167
168
169
170
171
172
173 public static Message obtain(Message orig) {
174 Message m = obtain();
175 m.what = orig.what;
176 m.arg1 = orig.arg1;
177 m.arg2 = orig.arg2;
178 m.obj = orig.obj;
179 m.replyTo = orig.replyTo;
180 m.sendingUid = orig.sendingUid;
181 m.workSourceUid = orig.workSourceUid;
182 if (orig.data != null) {
183 m.data = new Bundle(orig.data);
184 }
185 m.target = orig.target;
```

```
158 m.callback = orig.callback;
159
160 return m;
161 }
162
163
164
165
166
167
168 public static Message obtain(Handler h) {
169 Message m = obtain();
170 m.target = h;
171
172 return m;
173 }
174
175
176
177
178
179
180
181
182 public static Message obtain(Handler h, Runnable callback) {
183 Message m = obtain();
184 m.target = h;
185 m.callback = callback;
186
187 return m;
188 }
189
190
191
192
193
194
195
196
197 public static Message obtain(Handler h, int what) {
198 Message m = obtain();
199 m.target = h;
200 m.what = what;
201
202 return m;
203 }
204
205
```

```
186 m.callback = orig.callback;
187
188 return m;
189 }
190
191
192
193
194
195
196 public static Message obtain(Handler h) {
197 Message m = obtain();
198 m.target = h;
199
200 return m;
201 }
202
203
204
205
206
207
208
209
210 public static Message obtain(Handler h, Runnable callback) {
211 Message m = obtain();
212 m.target = h;
213 m.callback = callback;
214
215 return m;
216 }
217
218
219
220
221
222
223
224
225 public static Message obtain(Handler h, int what) {
226 Message m = obtain();
227 m.target = h;
228 m.what = what;
229
230 return m;
231 }
232
233
```

```
206
207
208
209
210
211
212
213 public static Message obtain(Handler h, int what, Object obj) {
214 Message m = obtain();
215 m.target = h;
216 m.what = what;
217 m.obj = obj;
218
219 return m;
220 }
221
222
223
224
225
226
227
228
229
230
231
232 public static Message obtain(Handler h, int what, int arg1, int arg2) {
233 Message m = obtain();
234 m.target = h;
235 m.what = what;
236 m.arg1 = arg1;
237 m.arg2 = arg2;
238
239 return m;
240 }
241
242
243
244
245
246
247
248
249
250
251
252
253 public static Message obtain(Handler h, int what,
```

```
234
235
236
237
238
239
240
241 public static Message obtain(Handler h, int what, Object obj) {
242 Message m = obtain();
243 m.target = h;
244 m.what = what;
245 m.obj = obj;
246
247 return m;
248 }
249
250
251
252
253
254
255
256
257
258
259
260 public static Message obtain(Handler h, int what, int arg1, int arg2) {
261 Message m = obtain();
262 m.target = h;
263 m.what = what;
264 m.arg1 = arg1;
265 m.arg2 = arg2;
266
267 return m;
268 }
269
270
271
272
273
274
275
276
277
278
279
280
281 public static Message obtain(Handler h, int what,
```

```
254 int arg1, int arg2, Object obj) {
255 Message m = obtain();
256 m.target = h;
257 m.what = what;
258 m.arg1 = arg1;
259 m.arg2 = arg2;
260 m.obj = obj;
261
262 return m;
263 }
264
265
266 public static void updateCheckRecycle(int targetSdkVersion) {
267 if (targetSdkVersion < Build.VERSION_CODES.LOLLIPOP) {
268 gCheckRecycle = false;
269 }
270 }
271
272
273
274
275
276
277
278
279
280 public void recycle() {
281 if (isInUse()) {
282 if (gCheckRecycle) {
283 throw new IllegalStateException("This message cannot be recycled because it "
284 + "is still in use.");
285 }
286 return;
287 }
288 recycleUnchecked();
289 }
290
291
292
293
294
295 void recycleUnchecked() {
296
297
298 flags = FLAG_IN_USE;
299 what = 0;
300 arg1 = 0;
```

```
282 int arg1, int arg2, Object obj) {
283 Message m = obtain();
284 m.target = h;
285 m.what = what;
286 m.arg1 = arg1;
287 m.arg2 = arg2;
288 m.obj = obj;
289
290 return m;
291 }
292
293
294 public static void updateCheckRecycle(int targetSdkVersion) {
295 if (targetSdkVersion < Build.VERSION_CODES.LOLLIPOP) {
296 gCheckRecycle = false;
297 }
298 }
299
300
301
302
303
304
305
306
307
308 public void recycle() {
309 if (isInUse()) {
310 if (gCheckRecycle) {
311 throw new IllegalStateException("This message cannot be recycled because it "
312 + "is still in use.");
313 }
314 return;
315 }
316 recycleUnchecked();
317 }
318
319
320
321
322
323 @UnsupportedAppUsage
324 void recycleUnchecked() {
325
326
327 flags = FLAG_IN_USE;
328 what = 0;
329 arg1 = 0;
```

```
301 arg2 = 0;
302 obj = null;
303 replyTo = null;
304 sendingUid = -1;
305 when = 0;
306 target = null;
307 callback = null;
308 data = null;
309
310 synchronized (sPoolSync) {
311 if (sPoolSize < MAX_POOL_SIZE) {
312 next = sPool;
313 sPool = this;
314 sPoolSize++;
315 }
316 }
317 }
318
319
320
321
322
323
324 public void copyFrom(Message o) {
325 this.flags = o.flags & ~FLAGS_TO_CLEAR_ON_COPY_FROM;
326 this.what = o.what;
327 this.arg1 = o.arg1;
328 this.arg2 = o.arg2;
329 this.obj = o.obj;
330 this.replyTo = o.replyTo;
331 this.sendingUid = o.sendingUid;
332
333 if (o.data != null) {
334 this.data = (Bundle) o.data.clone();
335 } else {
336 this.data = null;
337 }
338 }
339
340
341
342
343 public long getWhen() {
344 return when;
345 }
346
```

```
330 arg2 = 0;
331 obj = null;
332 replyTo = null;
333 sendingUid = UID_NONE;
334 workSourceUid = UID_NONE;
335 when = 0;
336 target = null;
337 callback = null;
338 data = null;
339
340 synchronized (sPoolSync) {
341 if (sPoolSize < MAX_POOL_SIZE) {
342 next = sPool;
343 sPool = this;
344 sPoolSize++;
345 }
346 }
347 }
348
349
350
351
352
353
354 public void copyFrom(Message o) {
355 this.flags = o.flags & ~FLAGS_TO_CLEAR_ON_COPY_FROM;
356 this.what = o.what;
357 this.arg1 = o.arg1;
358 this.arg2 = o.arg2;
359 this.obj = o.obj;
360 this.replyTo = o.replyTo;
361 this.sendingUid = o.sendingUid;
362 this.workSourceUid = o.workSourceUid;
363
364 if (o.data != null) {
365 this.data = (Bundle) o.data.clone();
366 } else {
367 this.data = null;
368 }
369 }
370
371
372
373
374 public long getWhen() {
375 return when;
376 }
377
```

```
347 public void setTarget(Handler target) {
348 this.target = target;
349 }
350
351
352
353
354
355
356
357
358
359 public Handler getTarget() {
360 return target;
361 }
362
363
364
365
366
367
368
369
370
371 public Runnable getCallback() {
372 return callback;
373 }
374
375
    public Message setCallback(Runnable r) {
376 callback = r;
377 return this;
378 }
379
380
381
382
383
384
385
386
387
388
389
390
391
392 public Bundle getData() {
393 if (data == null) {
```

```
378 public void setTarget(Handler target) {
379 this.target = target;
380 }
381
382
383
384
385
386
387
388
389
390 public Handler getTarget() {
391 return target;
392 }
393
394
395
396
397
398
399
400
401
402 public Runnable getCallback() {
403 return callback;
404 }
405
406
407 @UnsupportedAppUsage
408 public Message setCallback(Runnable r) {
409 callback = r;
410 return this;
411 }
412
413
414
415
416
417
418
419
420
421
422
423
424 public Bundle getData() {
425 if (data == null) {
```

```
394 data = new Bundle();
395 }
396
397 return data;
398 }
399
400
401
402
403
404
405
406
407 public Bundle peekData() {
408 return data;
409 }
410
411
412
413
414
415
416
417 public void setData(Bundle data) {
418 this.data = data;
419 }
420
421
422
423
424
425
426 public Message setWhat(int what) {
427 this.what = what;
428 return this;
429 }
430
431
432
433
434
435 public void sendToTarget() {
436 target.sendMessage(this);
437 }
438
439
440
441
```

```
426 data = new Bundle();
427 }
428
429 return data;
430 }
431
432
433
434
435
436
437
438
439 public Bundle peekData() {
440 return data;
441 }
442
443
444
445
446
447
448
449 public void setData(Bundle data) {
450 this.data = data;
451 }
452
453
454
455
456
457
458 public Message setWhat(int what) {
459 this.what = what;
460 return this;
461 }
462
463
464
465
466
467 public void sendToTarget() {
468 target.sendMessage(this);
469 }
470
471
472
473
```

```
442
443
444
445
446
447 public boolean isAsynchronous() {
448 return (flags & FLAG_ASYNCHRONOUS) != 0;
449 }
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477 public void setAsynchronous(boolean async) {
478 if (async) {
479 flags |= FLAG_ASYNCHRONOUS;
480 } else {
481 flags &= ~FLAG_ASYNCHRONOUS;
482 }
483 }
484
485 boolean isInUse() {
486 return ((flags & FLAG_IN_USE) == FLAG_IN_USE);
487 }
488
```

```
474
475
476
477
478
479 public boolean isAsynchronous() {
480 return (flags & FLAG_ASYNCHRONOUS) != 0;
481 }
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509 public void setAsynchronous(boolean async) {
510 if (async) {
511 flags |= FLAG_ASYNCHRONOUS;
512 } else {
513 flags &= ~FLAG_ASYNCHRONOUS;
514 }
515 }
516
517 boolean isInUse() {
518 return ((flags & FLAG_IN_USE) == FLAG_IN_USE);
519 }
520
521 @UnsupportedAppUsage
```

```java
void markInUse() {
flags |= FLAG_IN_USE;
}


public Message() {
}

@Override
public String toString() {
return toString(SystemClock.uptimeMillis());
}

String toString(long now) {
StringBuilder b = new StringBuilder();
b.append("{ when=");
TimeUtils.formatDuration(when - now, b);

if (target != null) {
if (callback != null) {
b.append(" callback=");
b.append(callback.getClass().getName());
} else {
b.append(" what=");
b.append(what);
}

if (arg1 != 0) {
b.append(" arg1=");
b.append(arg1);
}

if (arg2 != 0) {
b.append(" arg2=");
b.append(arg2);
}

if (obj != null) {
b.append(" obj=");
b.append(obj);
}

b.append(" target=");
b.append(target.getClass().getName());
} else {
b.append(" barrier=");
```

```java
void markInUse() {
flags |= FLAG_IN_USE;
}


public Message() {
}

@Override
public String toString() {
return toString(SystemClock.uptimeMillis());
}

@UnsupportedAppUsage
String toString(long now) {
StringBuilder b = new StringBuilder();
b.append("{ when=");
TimeUtils.formatDuration(when - now, b);

if (target != null) {
if (callback != null) {
b.append(" callback=");
b.append(callback.getClass().getName());
} else {
b.append(" what=");
b.append(what);
}

if (arg1 != 0) {
b.append(" arg1=");
b.append(arg1);
}

if (arg2 != 0) {
b.append(" arg2=");
b.append(arg2);
}

if (obj != null) {
b.append(" obj=");
b.append(obj);
}

b.append(" target=");
b.append(target.getClass().getName());
} else {
b.append(" barrier=");
```

```
536 b.append(arg1);
537 }
538
539 b.append(" }");
540 return b.toString();
541 }
542
543 void writeToProto(ProtoOutputStream proto, long fieldId) {
544 final long messageToken = proto.start(fieldId);
545 proto.write(MessageProto.WHEN, when);
546
547 if (target != null) {
548 if (callback != null) {
549 proto.write(MessageProto.CALLBACK, callback.getClass().getName());
550 } else {
551 proto.write(MessageProto.WHAT, what);
552 }
553
554 if (arg1 != 0) {
555 proto.write(MessageProto.ARG1, arg1);
556 }
557
558 if (arg2 != 0) {
559 proto.write(MessageProto.ARG2, arg2);
560 }
561
562 if (obj != null) {
563 proto.write(MessageProto.OBJ, obj.toString());
564 }
565
566 proto.write(MessageProto.TARGET, target.getClass().getName());
567 } else {
568 proto.write(MessageProto.BARRIER, arg1);
569 }
570
571 proto.end(messageToken);
572 }
573
574 public static final Parcelable.Creator<Message> CREATOR
575 = new Parcelable.Creator<Message>() {
576 public Message createFromParcel(Parcel source) {
577 Message msg = Message.obtain();
578 msg.readFromParcel(source);
579 return msg;
580 }
581
582 public Message[] newArray(int size) {
583 return new Message[size];
```

```
570 b.append(arg1);
571 }
572
573 b.append(" }");
574 return b.toString();
575 }
576
577 void dumpDebug(ProtoOutputStream proto, long fieldId) {
578 final long messageToken = proto.start(fieldId);
579 proto.write(MessageProto.WHEN, when);
580
581 if (target != null) {
582 if (callback != null) {
583 proto.write(MessageProto.CALLBACK, callback.getClass().getName());
584 } else {
585 proto.write(MessageProto.WHAT, what);
586 }
587
588 if (arg1 != 0) {
589 proto.write(MessageProto.ARG1, arg1);
590 }
591
592 if (arg2 != 0) {
593 proto.write(MessageProto.ARG2, arg2);
594 }
595
596 if (obj != null) {
597 proto.write(MessageProto.OBJ, obj.toString());
598 }
599
600 proto.write(MessageProto.TARGET, target.getClass().getName());
601 } else {
602 proto.write(MessageProto.BARRIER, arg1);
603 }
604
605 proto.end(messageToken);
606 }
607
608 public static final @android.annotation.NonNull Parcelable.Creator<Message> CREATOR
609 = new Parcelable.Creator<Message>() {
610 public Message createFromParcel(Parcel source) {
611 Message msg = Message.obtain();
612 msg.readFromParcel(source);
613 return msg;
614 }
615
616 public Message[] newArray(int size) {
617 return new Message[size];
```

```
584 }
585 };
586
587 public int describeContents() {
588 return 0;
589 }
590
591 public void writeToParcel(Parcel dest, int flags) {
592 if (callback != null) {
593 throw new RuntimeException(
594 "Can't marshal callbacks across processes.");
595 }
596 dest.writeInt(what);
597 dest.writeInt(arg1);
598 dest.writeInt(arg2);
599 if (obj != null) {
600 try {
601 Parcelable p = (Parcelable)obj;
602 dest.writeInt(1);
603 dest.writeParcelable(p, flags);
604 } catch (ClassCastException e) {
605 throw new RuntimeException(
606 "Can't marshal non-Parcelable objects across processes.");
607 }
608 } else {
609 dest.writeInt(0);
610 }
611 dest.writeLong(when);
612 dest.writeBundle(data);
613 Messenger.writeMessengerOrNullToParcel(replyTo, dest);
614 dest.writeInt(sendingUid);
615 }
616
617 private void readFromParcel(Parcel source) {
618 what = source.readInt();
619 arg1 = source.readInt();
620 arg2 = source.readInt();
621 if (source.readInt() != 0) {
622 obj = source.readParcelable(getClass().getClassLoader());
623 }
624 when = source.readLong();
625 data = source.readBundle();
626 replyTo = Messenger.readMessengerOrNullFromParcel(source);
627 sendingUid = source.readInt();
628 }
629 }
```

```
618 }
619 };
620
621 public int describeContents() {
622 return 0;
623 }
624
625 public void writeToParcel(Parcel dest, int flags) {
626 if (callback != null) {
627 throw new RuntimeException(
628 "Can't marshal callbacks across processes.");
629 }
630 dest.writeInt(what);
631 dest.writeInt(arg1);
632 dest.writeInt(arg2);
633 if (obj != null) {
634 try {
635 Parcelable p = (Parcelable)obj;
636 dest.writeInt(1);
637 dest.writeParcelable(p, flags);
638 } catch (ClassCastException e) {
639 throw new RuntimeException(
640 "Can't marshal non-Parcelable objects across processes.");
641 }
642 } else {
643 dest.writeInt(0);
644 }
645 dest.writeLong(when);
646 dest.writeBundle(data);
647 Messenger.writeMessengerOrNullToParcel(replyTo, dest);
648 dest.writeInt(sendingUid);
649 dest.writeInt(workSourceUid);
650 }
651
652 private void readFromParcel(Parcel source) {
653 what = source.readInt();
654 arg1 = source.readInt();
655 arg2 = source.readInt();
656 if (source.readInt() != 0) {
657 obj = source.readParcelable(getClass().getClassLoader());
658 }
659 when = source.readLong();
660 data = source.readBundle();
661 replyTo = Messenger.readMessengerOrNullFromParcel(source);
662 sendingUid = source.readInt();
663 workSourceUid = source.readInt();
664 }
665 }
```